

Hosting the Shero WhatsApp ordering bot on AWS

A step-by-step guide for the server team. It sets up one database (Amazon RDS) and one server (Amazon EC2) running the bot in Docker, with HTTPS. Expect about 1 to 2 hours.

What you are hosting

A Python web service (FastAPI) that receives WhatsApp messages from Gallabox, serves the customer ordering page and the admin dashboard, takes payments through Stripe, books Uber Direct couriers and records orders in Zoho CRM. It also runs timed jobs: payment reminders, link expiry, kitchen alerts and courier booking.

Part	AWS service	Holds / does
Database	Amazon RDS for PostgreSQL 16	Customers, orders, menu, kitchens
Server	Amazon EC2, Ubuntu 24.04, t3.small	Runs the bot and Caddy in Docker
HTTPS	Caddy (in Docker, on the same server)	Gets and renews a free certificate automatically
Dish photos	The server's disk (folder data/media)	About 30 MB today

The one rule that matters

Run **exactly one copy** of the bot. It sends timed WhatsApp messages (payment reminders, kitchen alerts); two copies would send every message twice. Do not put it in an auto-scaling group with more than one server.

What the Shero team gives you

Item	Details
Access to the code	https://github.com/NagiPragalathan/food_ordering_bot_shero.git (ask the developer for read access)
The .env file	All the passwords and API keys. Sent separately through a password manager or other secure channel, never by email or chat.
Dish photos	A zip of the data/media folder (shero-media.zip)
The domain name	The address customers will use, for example order.sherohomefood.com. Called DOMAIN in this guide.

What you need

- An AWS account where you can create EC2, RDS and Elastic IPs.
- Access to the DNS settings of the domain (Route 53, GoDaddy, Cloudflare...).
- An SSH client.

1 Choose the AWS region

Use **US East (N. Virginia) us-east-1**: the business and its customers are in the US. Create everything below in this one region.

2 Create the security groups

EC2 → **Security Groups** → **Create security group**. Make two:

Name	Inbound rules	Used by
shero-web	SSH 22 from your office IP only (My IP) HTTP 80 from 0.0.0.0/0 and ::/0 HTTPS 443 from 0.0.0.0/0 and ::/0	The server
shero-db	PostgreSQL 5432 from the security group shero-web only	The database

Ports 80 and 443 must be open to everyone: WhatsApp (Gallabox) and Stripe call the bot from the internet, and Caddy uses port 80 to get the certificate. Leave outbound rules as "All traffic".

3 Create the database (RDS)

RDS → **Databases** → **Create database** → **Standard create**:

Setting on the AWS screen	Choose or type
Engine	PostgreSQL, version 16.x (latest 16)
Template	Production (or Dev/Test for a trial)
DB instance identifier	shero-bot-db
Master username	shero
Master password	A long password using only letters and numbers (no @ : / % # ?). Keep it for step 7.
Instance class	db.t4g.micro (enough to start; db.t4g.small for more traffic)
Storage	gp3, 20 GiB, storage autoscaling on
Availability	Single-AZ to start (Multi-AZ for higher uptime, about twice the cost)
Connectivity → VPC	The same VPC the server will use (the default VPC is fine)
Public access	No
VPC security group	Choose existing: shero-db (remove "default")
Additional configuration → Initial database name	shero_bot
Backup retention	7 days

Setting on the AWS screen	Choose or type
Deletion protection	On

Click **Create database**. When its status is **Available** (about 10 minutes), open it and copy the **Endpoint**, which looks like `shero-bot-db.xxxxxxxx.us-east-1.rds.amazonaws.com`.

4 Create the server (EC2)

EC2 → **Instances** → **Launch instances**:

Setting on the AWS screen	Choose or type
Name	shero-bot
Image (AMI)	Ubuntu Server 24.04 LTS (64-bit x86)
Instance type	t3.small (2 GB memory)
Key pair	Create or choose one; you need it to log in
Network settings	Same VPC as the database; Auto-assign public IP: Enable
Security group	Select existing: shero-web
Storage	30 GiB gp3

Then give it a fixed address: EC2 → **Elastic IPs** → **Allocate Elastic IP address** → **Allocate**, then **Actions** → **Associate Elastic IP address** → choose the instance shero-bot. Note the IP address.

5 Point the domain at the server

In the domain's DNS settings, add one record:

Type	Name	Value	TTL
A	DOMAIN (for example order.sherohomefood.com)	The Elastic IP from step 4	300

Check it before going on (it can take a few minutes). The answer must be the Elastic IP:

```
nslookup DOMAIN
```

6 Log in and install Docker

```
ssh -i your-key.pem ubuntu@ELASTIC_IP

sudo apt-get update && sudo apt-get -y upgrade
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker ubuntu
exit                # log out and back in so the group applies

ssh -i your-key.pem ubuntu@ELASTIC_IP
docker compose version    # must print a version
timedatectl status        # must say: System clock synchronized: yes
```

The clock matters: Stripe rejects payment notifications from a server whose clock is wrong. Ubuntu keeps it synced by default.

7 Get the code and the settings file

```
git clone https://github.com/NagiPragalathan/food_ordering_bot_shero.git shero-bot
cd shero-bot
```

Put the **.env** file from the Shero team at **~/shero-bot/.env** (for example with scp from your computer), then lock it down:

```
scp -i your-key.pem .env ubuntu@ELASTIC_IP:~/shero-bot/.env    # from your computer
chmod 600 ~/shero-bot/.env                                     # on the server
```

Open it (nano ~/shero-bot/.env) and set these lines. Replace DOMAIN, PASSWORD and RDS_ENDPOINT with your values; leave every other line as it is:

```
APP_ENV=production
ENABLE_SCHEDULER=true
DOMAIN=order.sherohomefood.com
PUBLIC_BASE_URL=https://order.sherohomefood.com
PAY_REDIRECT_BASE_URL=https://order.sherohomefood.com/pay
DATABASE_URL=postgresql+asyncpg://shero:PASSWORD@RDS_ENDPOINT:5432/shero_bot?ssl=require
```

Important

- **?ssl=require** at the end is needed: RDS only accepts encrypted connections.
- Never commit .env to Git or paste it into a ticket. It holds payment and API keys.
- Do not change **SETTINGS_ENCRYPTION_KEY** or **ADMIN_SESSION_SECRET** later: settings saved in the dashboard and admin logins depend on them.

8 Add the dish photos

```
scp -i your-key.pem shero-media.zip ubuntu@ELASTIC_IP:~/ # from your computer

sudo apt-get install -y unzip # on the server
mkdir -p ~/shero-bot/data/media
unzip ~/shero-media.zip -d ~/shero-bot/data/media
sudo chown -R 10001:10001 ~/shero-bot/data/media
```

The bot runs as user id 10001 inside Docker, so it must own this folder. If there is no photo zip yet, still create the folder and run the chown line.

9 Start the bot

```
cd ~/shero-bot
docker compose -f deploy/docker-compose.prod.yml --env-file .env up -d --build
```

This builds the bot, creates the database tables, starts it, and starts Caddy, which gets the HTTPS certificate for DOMAIN. Both restart by themselves after a reboot or a crash. Watch the start-up:

```
docker compose -f deploy/docker-compose.prod.yml logs -f
# good signs: 'Running upgrade ...' then 'Application startup complete'
#             caddy: 'certificate obtained successfully'
# Ctrl+C to stop watching (the bot keeps running)
```

10 Check it works

```
curl https://DOMAIN/health
# {"status":"ok","environment":"production"}

curl https://DOMAIN/health/readiness
# {"status":"ready", ... "database":"ok", ... "missing_credentials":[] ...}
```

- Open **https://DOMAIN/admin/** in a browser: the Shero admin login page must appear, with a padlock (valid HTTPS).
- Point any uptime or load-balancer health check at **/health**, not /health/readiness.

11 Send these details back to the Shero team

They finish the setup that is done in other services and in the dashboard:

Send	Why
The final address	https://DOMAIN
Confirmation that /health is OK	So they can switch the WhatsApp webhook over
Admin login works	They import the menu, add the kitchens and reconnect Zoho

For reference, the Shero team then: points the Gallabox and Stripe webhooks at

<https://DOMAIN/webhooks/gallabox> and [/webhooks/stripe](https://DOMAIN/webhooks/stripe), resubmits the WhatsApp templates with the new address, adds DOMAIN to the Google Maps browser key, and adds <https://DOMAIN/admin/settings/zoho/callback> to the Zoho API client.

Running it

Updating to a new version

```
cd ~/shero-bot
git pull
docker compose -f deploy/docker-compose.prod.yml --env-file .env up -d --build
```

Database changes are applied automatically on start. The bot is unavailable for about a minute during the restart; WhatsApp and Stripe retry their messages, so nothing is lost.

Everyday commands (run in ~/shero-bot)

Task	Command
See the logs	<code>docker compose -f deploy/docker-compose.prod.yml logs -f api</code>
Restart after editing .env	<code>docker compose -f deploy/docker-compose.prod.yml --env-file .env up -d</code>
Is it running?	<code>docker compose -f deploy/docker-compose.prod.yml ps</code>
Stop everything	<code>docker compose -f deploy/docker-compose.prod.yml down</code>

Backups

- **Database:** RDS automated backups (7 days) were turned on in step 3. Restore from RDS → the database → Maintenance & backups.
- **Dish photos and settings file:** take a daily snapshot of the server's disk. EC2 → **Lifecycle Manager** → Create lifecycle policy → EBS snapshot policy, target the shero-bot instance, every 24 hours, keep 7.

Monitoring

- A CloudWatch alarm on the EC2 **status check failed** metric, emailing the team.
- An uptime check on **<https://DOMAIN/health>** every 5 minutes (Route 53 health check, or any uptime monitor).
- Logs are JSON lines. Worth alerting on: **unhandled_flow_error** (a bug), **stripe_webhook_rejected** (wrong Stripe secret or server clock), **kitchen_alert_failed**, **uber_delivery_failed**, **zoho_ensure_record_failed** (if it keeps happening).

Rough monthly cost (us-east-1, on-demand)

Item	Approximate
EC2 t3.small	about \$15

Item	Approximate
RDS db.t4g.micro, 20 GB, Single-AZ	about \$15
Public IPv4 address	about \$4
Disk, snapshots, data transfer	a few dollars
Total	about \$35 to \$40 a month

Approximate list prices; check the AWS pricing calculator for current figures.

Troubleshooting

Problem	Fix
Browser shows a certificate error, or Caddy logs 'challenge failed'	DNS does not point at the Elastic IP yet, or port 80 is closed. Fix, then restart: <code>docker compose -f deploy/docker-compose.prod.yml restart caddy</code>
Logs say the database connection failed or timed out	Check DATABASE_URL (endpoint, password, ?ssl=require) and that shero-db allows 5432 from shero-web.
/health/readiness lists missing_credentials	Those .env lines are empty. Ask the Shero team for the values, then restart.
Dish photos are missing, or upload fails	Run: <code>sudo chown -R 10001:10001 ~/shero-bot/data/media</code>
Customers get two of every reminder	Two copies are running. Keep exactly one.
Stripe payments do not update orders	Check the server clock (timedatectl) and that STRIPE_WEBHOOK_SECRET matches Stripe.